IRELE

Advanced Measurement and Data Driven Modelling

Lab report

Authors:

Arico Amaury

Colot Emmeran

Gueye Sidy

Professor:

J. Lataire

TA:

F. Vandeputte

Academic year:

2025 - 2026

Contents

1 Lab 1: RLS with forgetting factor

1.1 Main steps

1. Measure the input and output signals from the DUT. The input is arbitrary and the time varying part of the system is due to a manual switch.
2. Make a first estimate of the parameters with an LLS estimator
3. Use the RLS estimator with forgetting factor to track the time-varying parameters of the system.
4. Deduce from the estimated parameters the time instants at which the system changes.

1.2 Time-varying RLS estimation

Starting from the recursive equation of the system, a matrix formulation is derived:

$$y(N) = b_0(N)u(N) + b_1(N)u(N-1) - a_1(N)y(N-1) - a_2(N)y(N-2) \tag{1.1}$$

$$= K(N)\theta(N) \tag{1.2}$$

with

$$K(N) = \begin{bmatrix} u(N) & u(N-1) & -y(N-1) & -y(N-2) \end{bmatrix}, \qquad \theta(N) = \begin{bmatrix} b_0(N) \\ b_1(N) \\ a_1(N) \\ a_2(N) \end{bmatrix} \tag{1.3}$$

The RLS estimator (with forgetting factor) is a recursive algorithm that computes the parameters $\theta(N)$ and their covariance matrix K_N after each new sample is collected. Because it is recursive, it needs to be initialized and this is done with a linear least squares estimate on a few first samples.
The linear least squares estimator, used to find θ in $y_{\text{lls}} = H\theta$, is in our case constructed as follows:

$$H = \begin{bmatrix} u(n_{\text{start}}+1) & u(n_{\text{start}}) & -y(n_{\text{start}}) & -y(n_{\text{start}}-1) \\ u(n_{\text{start}}+2) & u(n_{\text{start}}+1) & -y(n_{\text{start}}+1) & -y(n_{\text{start}}) \\ \vdots & \vdots & \vdots & \vdots \\ u(n_{\text{LLS}}-1) & u(n_{\text{LLS}}-2) & -y(n_{\text{LLS}}-2) & -y(n_{\text{LLS}}-3) \end{bmatrix}, \qquad y_{\text{lls}} = \begin{bmatrix} y(n_{\text{start}}+1) \\ y(n_{\text{start}}+2) \\ \vdots \\ y(n_{\text{LLS}}-1) \end{bmatrix} \tag{1.4}$$

Where $n_{\text{start}} = \max(n_a, n_b)$ and n_{LLS} is the number of samples used for the LLS estimate, chosen to be 200 here.

$\theta(n_{\text{LLS}})$ and $P_{n_{\text{LLS}}}$ are found with, respectively, the solution to the LLS problem and the definition of the covariance matrix in the RLS algorithm:

$$\theta(n_{\text{LLS}}) = (H^T H)^{-1} H^T y_{\text{lls}}, \qquad P_{n_{\text{LLS}}} = (H^T H)^{-1} \tag{1.5}$$

The Matlab code of the LLS initialization is given below:

```
% initialization with an LLS estimate over a few of the first samples
% y_lls = H * theta
H = zeros(sampleForLLS - max(na, nb), na + nb + 1);
for uCol = 0:nb
    H(:, uCol + 1) = u((max(na, nb) + 1 - uCol:sampleForLLS - uCol));
end
for yCol = 1:na
    H(:, nb + 1 + yCol) = -y((max(na, nb) + 1 - yCol:sampleForLLS - yCol));
end
y_lls = y((max(na, nb) + 1:sampleForLLS));

theta = H\y_lls;
P = inv(H'*H);
```

The recursive least squares equations with forgetting factor g are given by:

$$\theta(N) = \theta(N-1) - \frac{P_{N-1}K^T(N)\left[K(N)\theta(N-1) - y(N)\right]}{g + K(N)P_{N-1}K^T(N)} \tag{1.6}$$

$$P_N = \frac{1}{g}\left(P_{N-1} - \frac{P_{N-1}K^T(N)K(N)P_{N-1}}{g + K(N)P_{N-1}K^T(N)}\right) \tag{1.7}$$

The Matlab code implementing these equations is given below:

```
theta_sav = zeros(na + nb + 1, N - max(na, nb) + 2);
P_diag_sav = zeros(na + nb + 1, N - max(na, nb) + 2);

for itr = sampleForLLS+1:N
```

```
    % saving previous values
    theta_sav(:, itr - max(na, nb) + 1) = theta;
    P_diag_sav(:, itr - max(na, nb) + 1) = diag(P);

    K_itr = [u(itr - (0:nb)), -y(itr - (1:na))];

    theta = theta - (P * (K_itr') * (K_itr * theta - y(itr))) / (g + K_itr * P * (K_itr'));
    P = 1/g * (P - (P * (K_itr') * K_itr * P) / (g + K_itr * P * (K_itr')));

end
```

1.3 Results

The input of the system is not interesting to show here as it is random white noise. The evolution in time of the estimated parameters is however much more interesting. The following figures will display the evolution of the parameters (with their uncertainty range) for mutliple forgetting factor values and for different experiments. The experiments are :

1. Excitation of a LTI sub-system (namely Low LTI) - Figure 1.1.
2. Excitation of the second LTI sub-system (namely High LTI) - Figure 1.2.
3. Excitation of both LTI systems trough a single switch - Figure 1.3.
4. Excitation of both LTI systems trough mutliple switches - Figure 1.4.

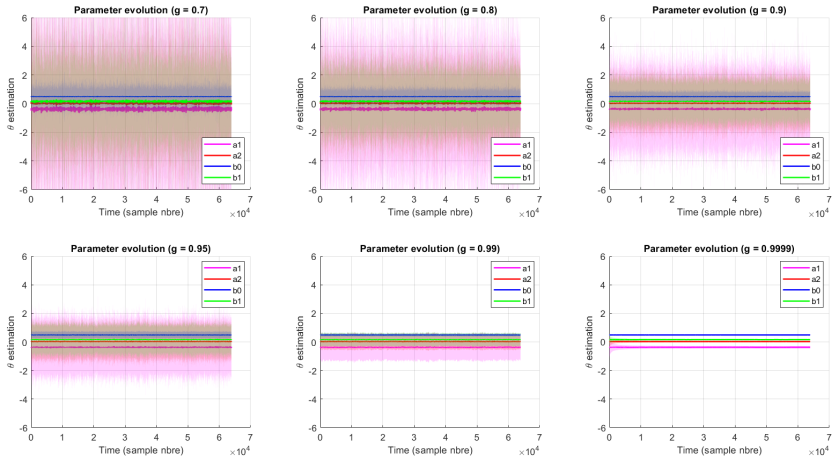

Figure 1.1: Parameters evolution for multiple forgetting factors - Low LTI sub-system

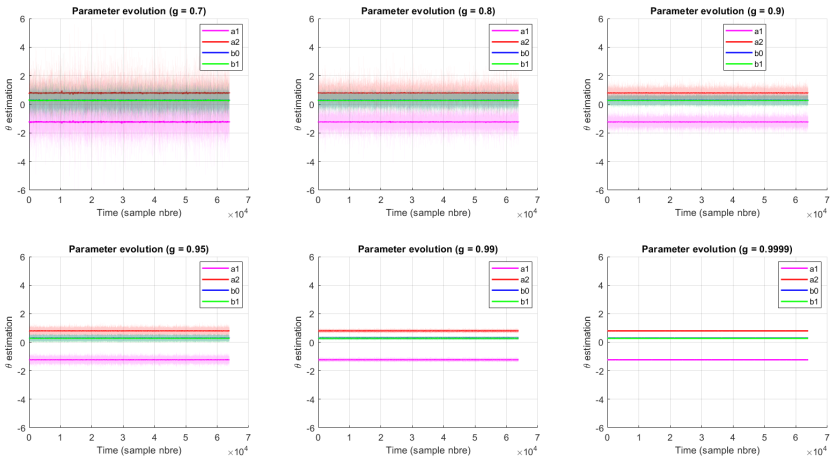

Figure 1.2: Parameters evolution for multiple forgetting factors - High LTI sub-system

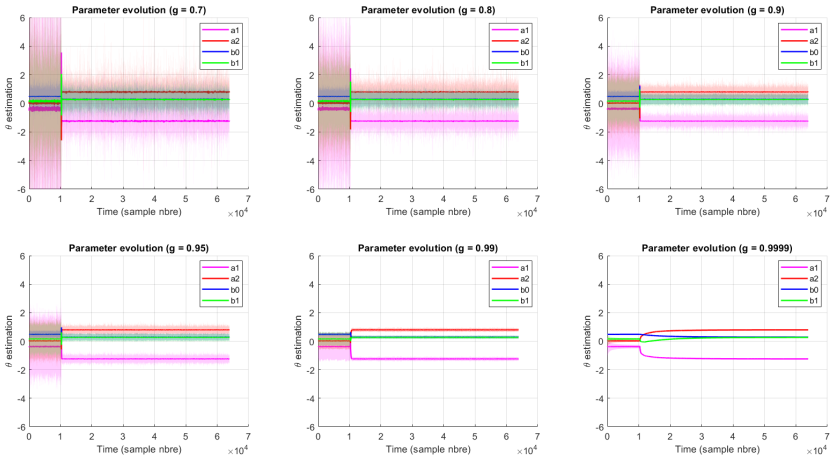

Figure 1.3: Parameters evolution for multiple forgetting factors - single switch between the two LTI systems

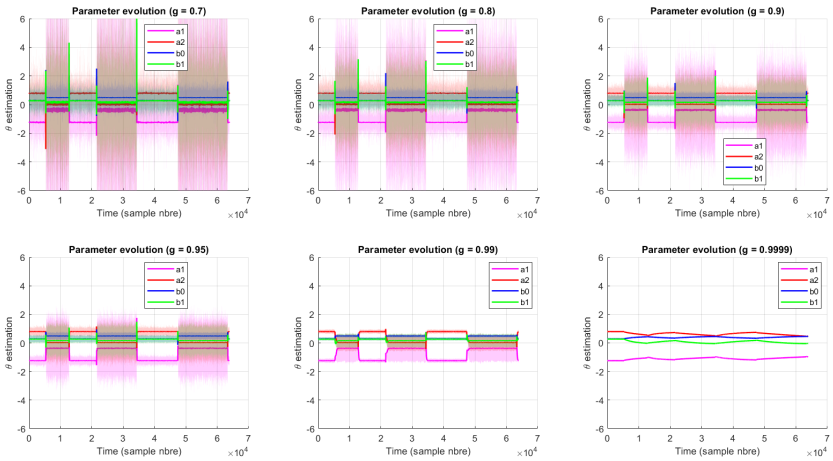

Figure 1.4: Parameters evolution for multiple forgetting factors - multiple switches between the two LTI systems

One may already observe that the uncertainty on the parameters of the LTI system named High LTI system is around 4 times lower than the uncertainty on the parameters for the second LTI system.
Another observation is related to the reduction of the uncertainty converging to a plateau value as the time passes at the beginning of the measurement. This behaviour is well visible in the case of the Low LTI subsystem for a forgetting factor set at 0.9999. The reason behind the uncertainty level diminution is laying in the limited number of samples available for the RLS algorithm at the beginning of the measurements - Figure 1.5.

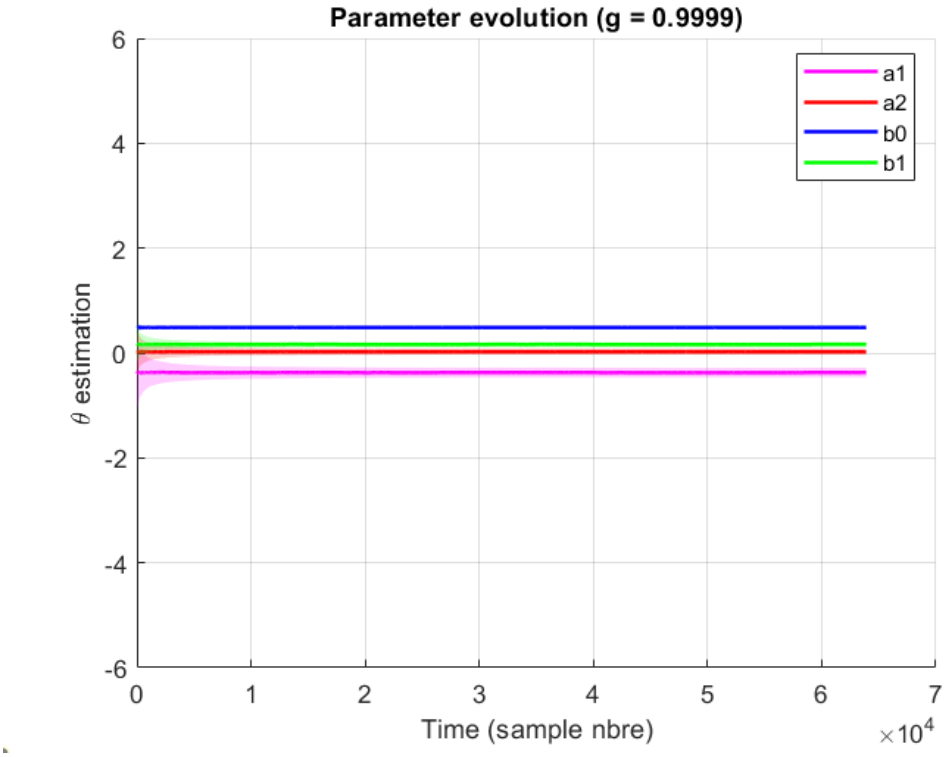

Figure 1.5: Uncertainty variation during the first 500 samples - Low LTI system (g=0.9999)

As shown on the figures, the RLS algorithm with forgetting factor is a powerful tool to track the parameters of time-varying system simulated in our case by switching two LTI subsystems. Nevertheless, the choice of the forgetting factor value is dependent of the constraints or specifications of our system and also the required confidence on the parameters we get once in steady-state.
Let's consider our experiment with mutliple switches - One can easily see that the RLS algorithm with a forgetting factor of 0.9999 cannot converge fast enough and is therefore not relevant for such time-varying system.
In the other hand, the other RLS algorithm with forgetting factor smaller than 0.9999 exhibits higher parameter variances leading to uncertainty much more significant (10 times higher in case of g = 0.99); but also noticeable spikes at the switching instants resulting to a non-smooth evolution of the parameters. This rough evolution may result to potential issues, damages in real systems we are tracking (in case by example

of RLS algorithm used for control system). For rigorosity, a table containing the parameters variance for the different forgetting factors is shown below - Table 1.1 and 1.2.

Low LTI	σ^2_{a1}	σ^2_{a2}	σ^2_{b0}	σ^2_{b1}
g=0.7	46.0729	19.1852	0.3077	13.5596
g=0.8	19.5318	8.1543	0.1653	5.5176
g=0.9	7.2197	2.7688	0.0767	1.9273
g=0.95	3.5704	1.2898	0.0393	0.9333
g=0.99	0.2875	0.1140	0.0090	0.0791
g=0.9999	$0.2385e^{-3}$	$0.2123e^{-3}$	$0.1125e^{-3}$	$0.1429e^{-3}$

Table 1.1: Parameters variances in steady state for Low LTI system

High LTI	σ^2_{a1}	σ^2_{a2}	σ^2_{b0}	σ^2_{b1}
g=0.7	1.2416	1.4443	2.0137	2.7133
g=0.8	0.5458	0.6222	0.7531	0.9635
g=0.9	0.1918	0.1995	0.2014	0.2356
g=0.95	0.0874	0.0842	0.0694	0.0798
g=0.99	0.0204	0.0189	0.0117	0.0135
g=0.9999	$0.4765e^{-3}$	$0.3578e^{-3}$	$0.1062e^{-3}$	$0.1921e^{-3}$

Table 1.2: Parameters variances in steady state for High LTI system

It remains therefore to decide the most suitable forgetting factor for our experiment with multiple switches. In our case, the forgetting factor of 0.99 was converging few hundreds of samples after the switch (0.15 seconds) to its steady-state - fast enough to converge before the next switching and with a smooth convergence. The switching instants and the convergence of the parameters to its steady-sate is shown in Figure 1.6.

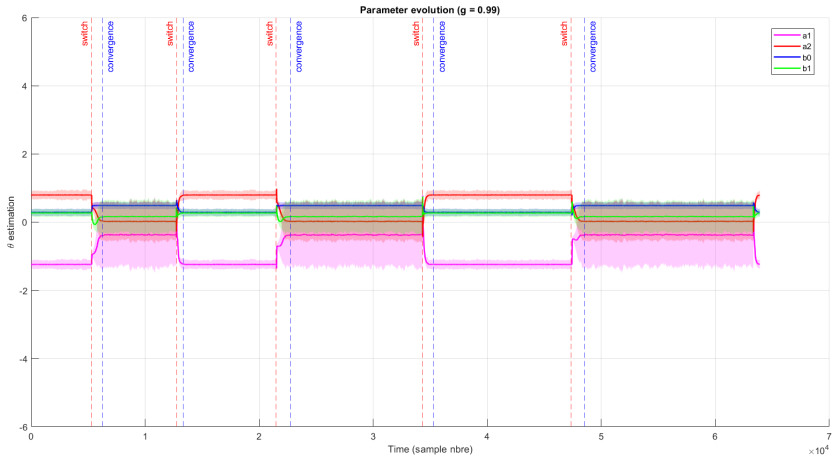

Figure 1.6: Switching instants and convergence (g=0.99)

In order to grasp the differences between the results given for different forgetting factor, it is useful to plot the transfer function modeled from the parameters given by the RLS algorithms and compare it with an estimate of the FRF computed using spectral averaging.

We can easily see that RLS algorithm with forgetting factor smaller than 0.99 have reached the convergence

and match the estimated FRF (which is logical as those algorithm are reacting faster than the algorithm with g=0.99). Nevertheless, by looking carefully to the modeled transfer functions coming from RLS with forgetting factors smaller than 0.99, one can observe the models are fitting the FRF with a lower accuracy resulting from the higher uncertainty level on the parameters.

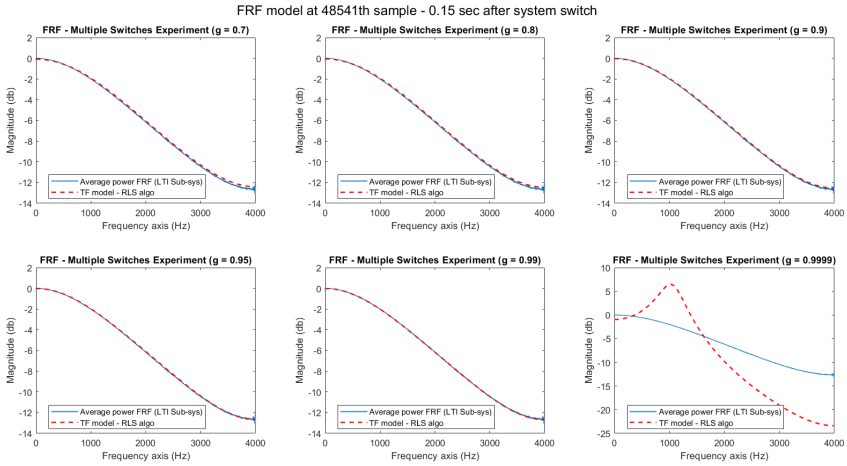

Figure 1.7: Comparison of the estimated FRF and the RLS based estimate after 150 ms

Finally, the comparison between the estimated FRF for both LTI sub-systems and the transfer function calculated based on the parameters - computed by the RLS algorithm with forgetting factor of 0.99 once the steady-state is reached - are displayed in Figure 1.8 and 1.9

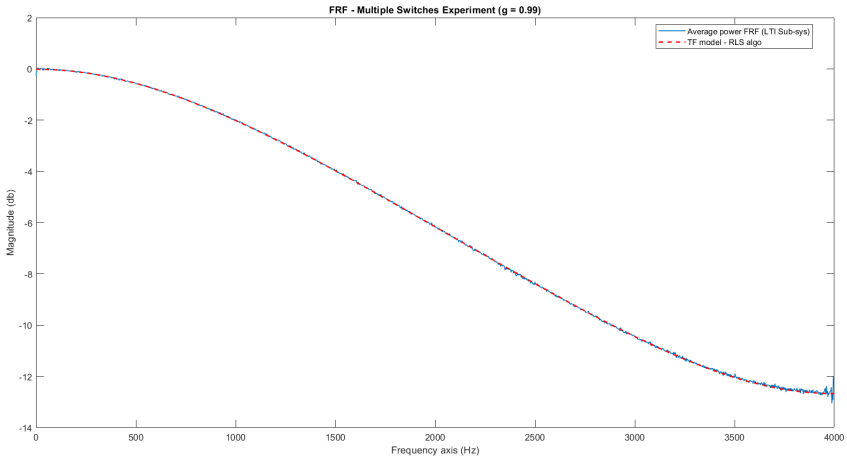

Figure 1.8: Comparison estimated FRF and computed model - Low LTI system - g=0.99

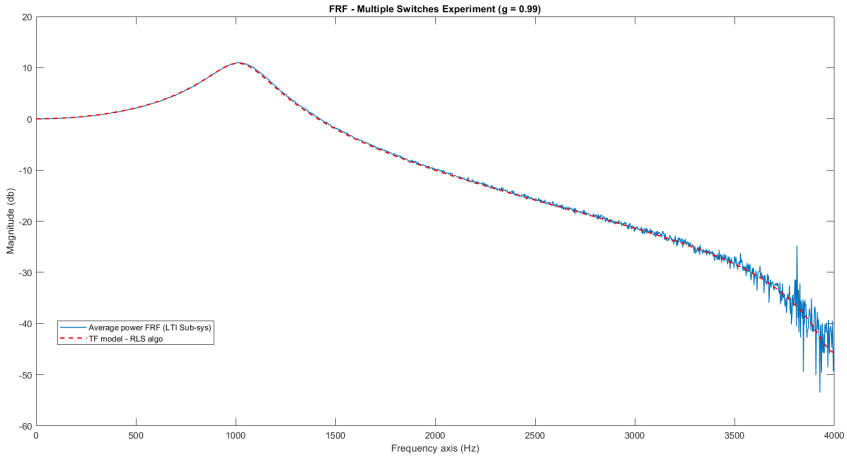

Figure 1.9: Comparison estimated FRF and computed model - High LTI system - g=0.99

Those parametric estimates (RLS - g=0.99 - steady-state instants) are detailed below:

$$G_{low}(z^{-1}) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{0.4874 + 0.1613 z^{-1}}{1 - 0.3763 z^{-1} + 0.0263 z^{-2}} \tag{1.8}$$

$$G_{high}(z^{-1}) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{0.2871 + 0.2713 z^{-1}}{1 - 1.2378 z^{-1} + 0.7951 z^{-2}} \tag{1.9}$$

In comparison the transfer function computed from LLS for each LTI sub-system taken separately :

$$G_{low-LLS}(z^{-1}) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{0.4832 + 0.1752 z^{-1}}{1 - 0.3518 z^{-1} + 0.01 z^{-2}} \tag{1.10}$$

$$G_{high-LLS}(z^{-1}) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1} + a_2 z^{-2}} = \frac{0.2884 + 0.2659 z^{-1}}{1 - 1.2419 z^{-1} + 0.7964 z^{-2}} \tag{1.11}$$

1.4 Conclusion

The RLS algorithm with forgetting factor is able to converge towards the true parameters of a time-varying system, under the condition that the forgetting factor is well chosen. This choice was easy with this system as it was known that it was switching between two LTI systems. In a more complex case, knowing whether the system is varying slowly or the forgetting factor is too high may be difficult.

2 Lab 2: Kalman filter

2.1 Main steps

1. TODO

2.2 Continuous-time state space model

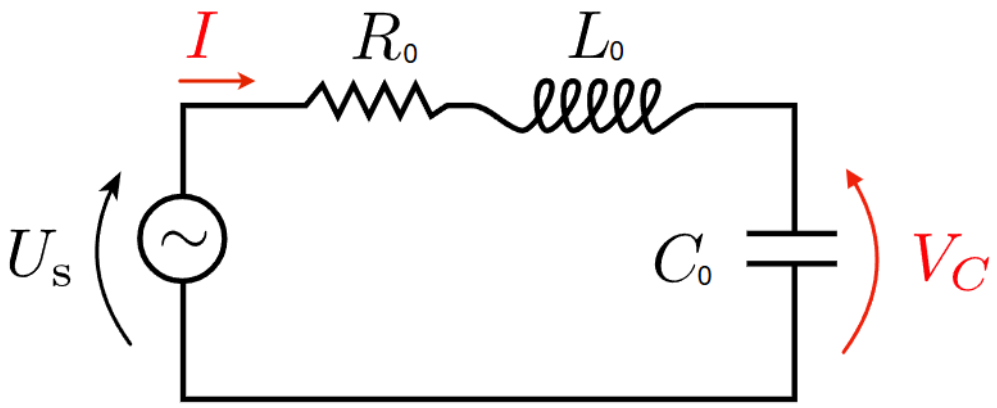

Figure 2.1: Studied RLC circuit

The studied system is the RLC circuit shown in Fig 2.1. A set of equations can be derived using Kirchhoff's laws:

$$u_s(t) = R_0\, i(t) + L_0 \frac{di(t)}{dt} + \frac{1}{C_0} \int i(t)\, dt \tag{2.1}$$

$$v_C(t) = \frac{1}{C_0} \int i(t)\, dt \tag{2.2}$$

By substituting the second equation into the first one, we obtain:

$$u_s(t) = R_0\, i(t) + L_0 \frac{di(t)}{dt} + v_C(t) \tag{2.3}$$

To get to the continuous-time state space representation of this circuit, we need to define the input of the system as $u_s(t)$, its output as $v_C(t)$ and the state vector as:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} v_C(t) \\ i(t) \end{bmatrix} \tag{2.4}$$

And using:

$$\begin{cases} \frac{dV_c(t)}{dt} = \frac{i(t)}{C_0} \\ \frac{di(t)}{dt} = -\frac{R_0}{L_0} i(t) - \frac{v_C(t)}{L_0} + \frac{u_s(t)}{L_0} \end{cases} \tag{2.5}$$

Which finally leads to:

$$\frac{d}{dt} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{C_0} \\ -\frac{1}{L_0} & -\frac{R_0}{L_0} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{L_0} \end{bmatrix} u_s(t) \tag{2.6}$$

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} \tag{2.7}$$

$$\Leftrightarrow \qquad A_c = \begin{bmatrix} 0 & \frac{1}{C_0} \\ -\frac{1}{L_0} & -\frac{R_0}{L_0} \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ \frac{1}{L_0} \end{bmatrix}, \qquad C_c = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D_c = 0 \tag{2.8}$$

2.3 Discrete-time state space model

To convert this continuous-time state space model to a discrete-time one, we need to select a sampling frequency. As the poles of the system are the eigenvalues of the matrix A_c, we can compute them with Matlab, giving:

$$\begin{cases} \lambda_1 \approx -1\text{MHz} + j8.1 \\ \lambda_2 \approx -1\text{MHz} - j8.1 \end{cases} \tag{2.9}$$

The band of interest is therefore chosen to be higher than 1 MHz in order to capture the dynamics of the system. the sampling frequency is thus set to $f_s = 4$ MHz. The discrete-time state space matrices are computed with Matlab. Two different methods are used: ZOH and tustin.
To compare which method is closer to the continuous-time model, the step response of the three models are plotted in Fig 2.2.

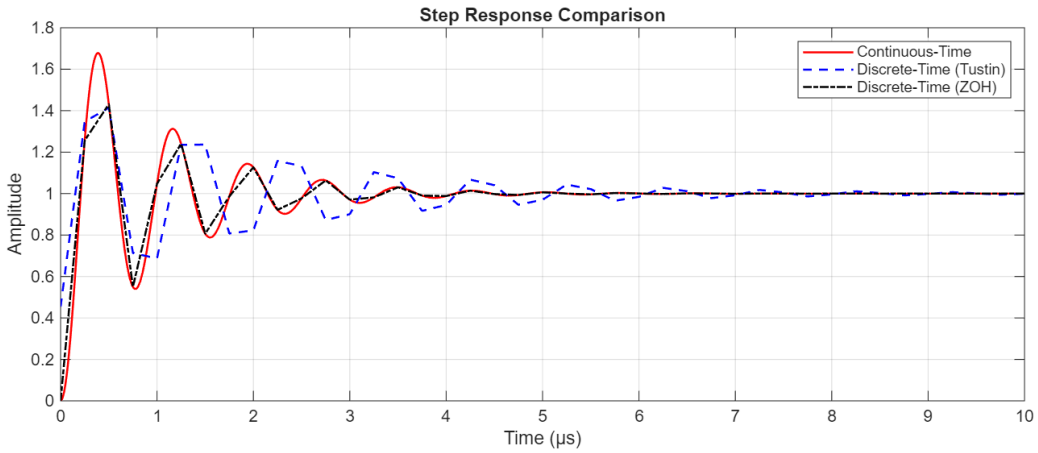

Figure 2.2: Step response comparison between continuous-time model and discrete-time models obtained with ZOH and Tustin methods

The tustin model oscillates for longer than the continuous-time model, which is not the case for the ZOH model. The ZOH model is therefore chosen for the following steps. The discrete-time state space matrices are:

$$A_d = \begin{bmatrix} -0.2560 & 28.7752 \\ -0.0173 & -0.4286 \end{bmatrix}, \qquad B_d = \begin{bmatrix} 1.2560 \\ 0.0173 \end{bmatrix}, \qquad C_d = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D_d = 0 \tag{2.10}$$

The poles of the discrete-time system are also computed by calculating the eigenvalues of the matrix A_d and the system is stable as they are in the unit circle:

$$\begin{cases} p_1 \approx -0.3423 + 0.6995i \\ p_2 \approx -0.3423 - 0.6995i \end{cases} \tag{2.11}$$

To determine whether the system is observable, we can compute the rank of the observability matrix $\mathcal{O}$:

$$\mathcal{O} = \begin{bmatrix} C_d \\ C_d A_d \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -0.2560 & 28.7752 \end{bmatrix} \tag{2.12}$$

Which is of rank 2, meaning that the system is observable.

2.4 Simulink simulation

The Simulink model is shown in Fig 2.3. The input is white Gaussian noise with zero mean and variance of 1V. There is also process noise $R_{v_{sim}}$ and measurement noise $R_{n_{sim}}$ which are respectively equal to:

$$R_{v_{sim}} = \begin{bmatrix} 0.025 & 0 \\ 0 & 10^{-6} \end{bmatrix}, \qquad R_{n_{sim}} = 0.0025 \tag{2.13}$$

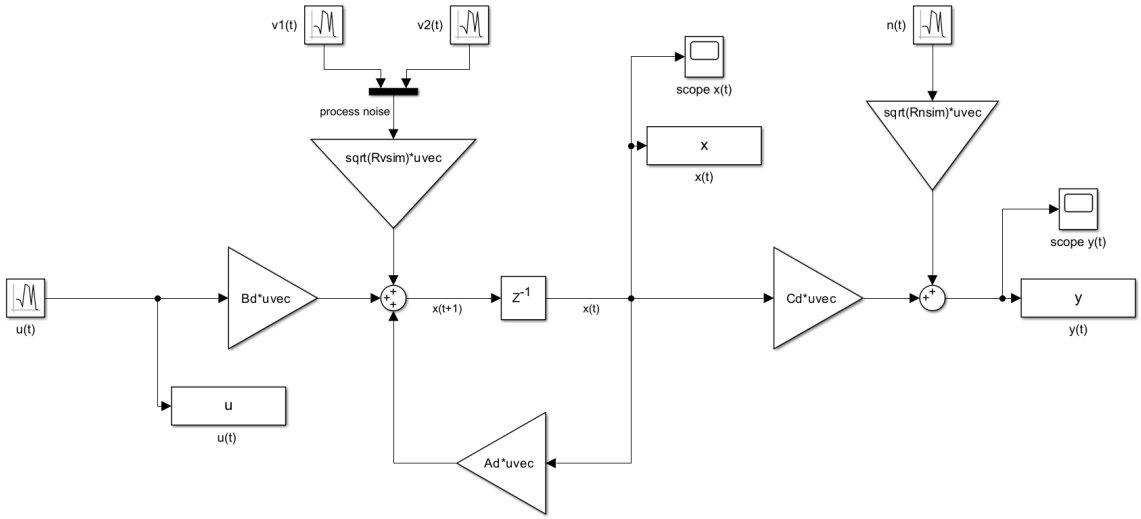

Figure 2.3: Simulink model of the discrete-time state space system

2.5 State estimation

The state estimation is performed in two ways: a simple prediction based on the model, and a Kalman filter. Both methods are implemented in Matlab as follows:

```
for itr = 1:size(xin, 1)-1
    % prediction only
    xPred(:, itr+1) = A * xPred(:, itr+1) + B * uin(itr, :);

    % Kalman
    Q = A * P(:, :, itr) * A' + Rv; % prediction covariance
    K(:, itr+1) = Q * C' / (C * Q * C' + Rn); % Kalman gain
    P(:, :, itr+1) = (eye(size(A, 1)) - K(:, itr+1) * C) * Q; % updated covariance

    prediction = A * xKalman(:, itr) + B * uin(itr, :);
    measurement = (yin(itr+1) - C*prediction - D*uin(itr+1, :));

    xKalman(:, itr+1) = prediction + K(:, itr+1) * measurement;
end
```

And those state estimates are compared to the true state, obtained from the Simulink model. The results are shown in Fig 2.4. It can be observed that the error on the state estimation is approximately 25 dB lower when using the Kalman filter compared to the simple prediction method.

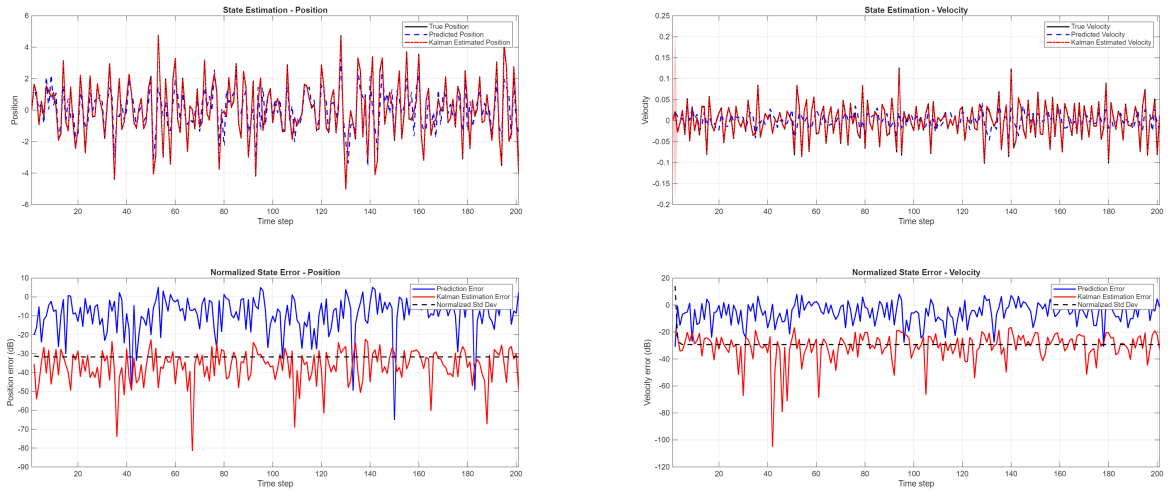

Figure 2.4: State estimation comparison between simple prediction and Kalman filter

To compare the robustness of both methods, the true parameters of the system are modified and the state estimation is performed again. Only the normalized state error plots are shown.

Fig 2.5 shows that an error on the model (here the A matrix) increases the state estimation error for both methods, but the Kalman filter still performs better than the simple prediction.

The impact of a wrong initial condition x_0 is shown in Fig 2.6. The Kalman filter has no issue converging to the true state, while the simple prediction is significantly affected.

Fig 2.7 and Fig 2.8 show the effect of wrong process and measurement noise covariances. In both cases, the estimates are not significantly affected, but the estimated standard deviation is higher than with the true parameters. It can be noted that it is 10 dB higher, which is consistent with the fact that the covariance matrices were multiplied by 100.

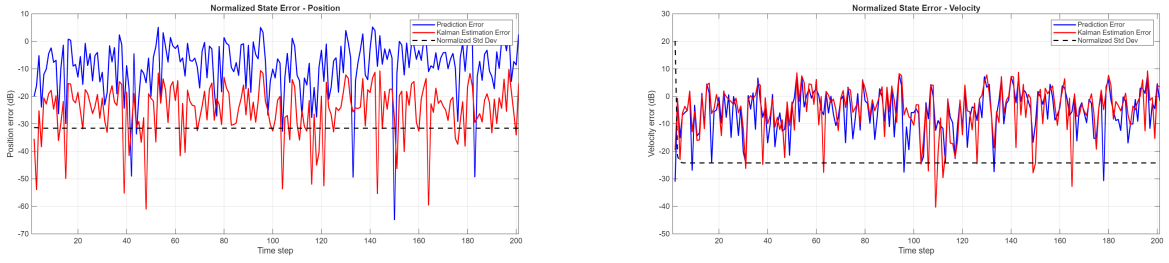

Figure 2.5: State estimation comparison with $A = 2A_{\text{true}}$

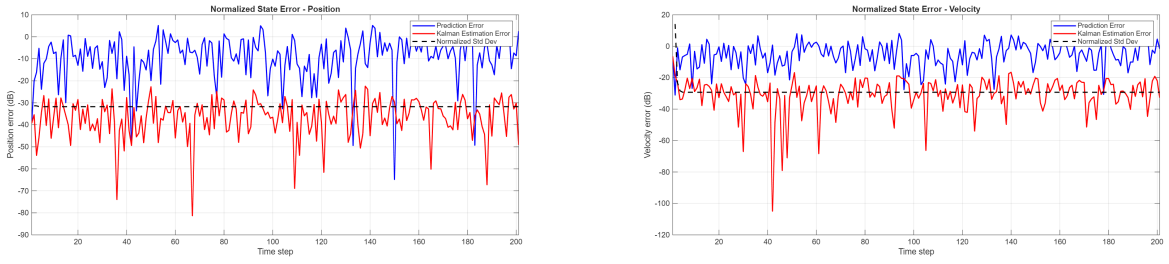

Figure 2.6: State estimation comparison with $x_0 = [0.02; 0.02]$ instead of $x_0 = [0; 0]$

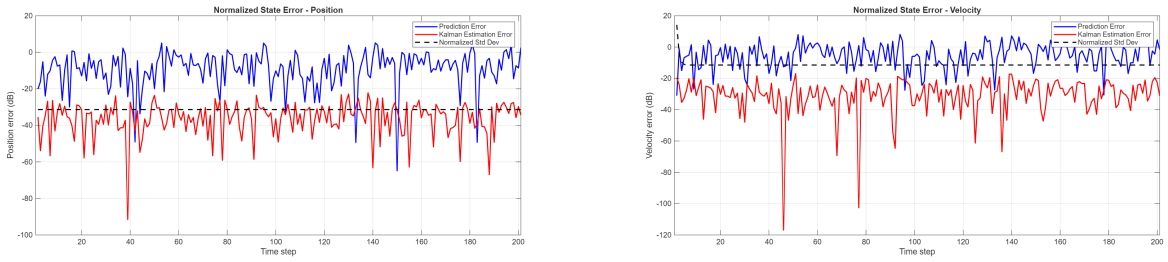

Figure 2.7: State estimation comparison with $R_v = 100R_{v_{\text{true}}}$

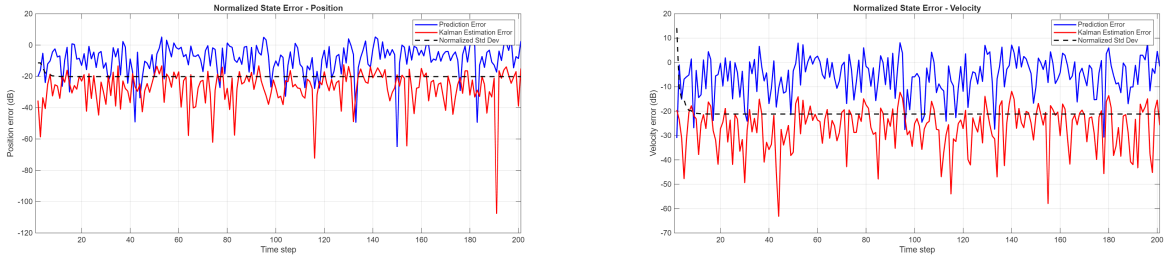

Figure 2.8: State estimation comparison with $R_n = 100R_{n_{\text{true}}}$

2.6 Conclusions

2.6.1 Which method performs better?

Method b (the Kalman filter) performs better. In the experiments run from matlab the Kalman filter produces much smaller state estimation error and converges to the true state independant of the initial condition, whereas the open-loop prediction remains biased and has larger steady-state error. (In our test runs the Kalman filter reduced the estimation error by a margin on the order of tens of dB relative to the open-loop prediction.)

2.6.2 Which method is more robust?

The Kalman filter is more robust in the presence of measurement noise and uncertain initial states: it adapts its estimates using incoming measurements and reduces bias that the predictor cannot correct. However, the Kalman filter is sensitive to model mismatch and to incorrect tuning of the noise covariances: large model errors degrade its performance (but typically it still outperforms the naive predictor).

In short, Kalman = more robust to noise and initial state, but still requires a reasonably accurate model and sensible covariance tuning.

2.6.3 Is there a difference in performance between the estimate of the first state (capacitor voltage) and of the second state (current) for method a? And for method b? Why?

2.6.4 What are the effects of the covariance matrices on the Kalman gain?

The Kalman gain K depends on the covariance of the predicted state Q, the measurement matrix C and the measurement noise covariance R_n via following relation

$$K_{t+1} = Q_{t+1} C^\top (C Q_{t+1} C^\top + R_n)^{-1}.$$

and where covariance on the estimated state P_{t+1} and covariance on the predicted state Q_{t+1} are computed using follwing relationships:

$$P_{t+1} = (I - K_{t+1} C) Q_{t+1}$$

$$Q_{t+1} = A P_t A^\top + R_v.$$

Thus, intuitively:

- Increasing the measurement noise R_n makes the denominator larger and therefore reduces K: the filter trusts the model more and the noisy measurements less.
- Increasing the covariance on the predicted state Q tends to increase K: larger uncertainty in the state estimate makes the filter rely more on measurements.
- Increasing the process noise covariance R_v increases Q and hence increases K.
- Initial P affects the transient gains.

Edge cases: as $R_n \to 0$ the gain tends to the (pseudo-)inverse, while as $R_n \to \infty$ the gain tends to zero and the filter behaves like a simple predictor.